Ministry of Higher Education and Scientific Research

University of Mustapha Benboulaid – Batna 2, Algeria

Faculty of Mathematics and Computer Science

Computer Science Department

MASTER 1 DTI

Speciality: Digital Transformation and Innovation

1st year Master · Semester 1

Module: Innovation Management

Practical Work – Mini Project 1 · Step 6: Documentation

TECHNICAL DOCUMENTATION

Innovation Idea Tracker

Code, architecture and setup instructions

Presented by

MEDKOUR Fadi

Master 1 DTI

Teacher

Mrs Karima Saidi

Innovation Management

Academic year 2026 – 2027

Université Batna 2 · November 2026

Table of Contents

1. Overview **3**
1.1 Purpose 3
1.2 Technology stack 3
1.3 Feature summary 3
2. Architecture **5**
2.1 Layered architecture 5
2.2 Application factory 5
2.3 Request life cycle 6
2.4 Blueprints 6
3. Project Structure **7**
4. Configuration **8**
5. Data Model **9**
5.1 Entity-relationship diagram 9
5.2 Tables 9
5.3 Integrity rules in the database 10
5.4 Computed columns 10
6. Code Walkthrough **11**
6.1 Authentication (blueprints/auth.py) 11
6.2 Role-based access control (utils.py) 11
6.3 Listing, searching and sorting (ideas.list_ideas) 12
6.4 Voting (ideas.vote) 12
6.5 Threaded comments 13
6.6 Evaluation workflow (ideas.review) 14
6.7 Administration (blueprints/admin.py) 14
6.8 Security module (security.py) 14
6.9 Front end 15
7. Routes **16**
8. Security Measures **17**
9. Installation and Setup **18**
9.1 Prerequisites 18
9.2 Local installation 18
9.3 Creating the first administrator 18
9.4 Demonstration accounts 19
9.5 Using PostgreSQL 19
9.6 Docker 19
9.7 Production server 19

10. Testing **20**
10.1 Running the tests 20
10.2 How the tests are organised 20
11. Deployment **21**
12. Extending the Application **22**
Known limitations 22
Annex – Inventory of automated tests **23**

1. Overview

1.1 Purpose

This document describes **how the Innovation Idea Tracker is built**: its architecture, the structure of the source code, the data model, the main algorithms, the security measures, and the exact steps to install, run, test and deploy it. It is intended for developers who must maintain or extend the application, and for evaluators who want to understand the implementation. End-user instructions are in the separate *User Guide*.

1.2 Technology stack

Table 1 – Technologies

Layer	Technology	Role
Language	Python 3.10+	Server-side code
Web framework	Flask 3.1	Routing, requests, templates, application factory, blueprints
ORM / database	Flask-SQLAlchemy 3.1, SQLAlchemy 2.x; SQLite (dev), PostgreSQL via `psycopg2` (prod)	Data access with one model for both databases
Authentication	Flask-Login 0.6, Werkzeug (scrypt)	Sessions, password hashing
Forms / CSRF	Flask-WTF 1.3, WTForms 3, email-validator	Validation and CSRF protection
Templates	Jinja2	HTML rendering with auto-escaping and macros
Front end	Bootstrap 5.3, Bootstrap Icons, Chart.js 4, Inter font (vendored), vanilla JavaScript	Responsive UI, charts, AJAX voting
Server	Gunicorn	Production WSGI server
Tests / CI	pytest, GitHub Actions, Playwright (browser checks)	Automated verification

1.3 Feature summary

- Registration, login, roles (submitter / reviewer / administrator) enforced on the server.
- Ideas with predefined and user-created categories; search, filters, five sort orders, pagination.
- Up/down voting (one vote per user and idea), threaded comments, evaluation workflow with review queue.
- Administration area (dashboard, users, categories, feedback, CSV export) and a feedback loop.

- Strict security configuration (CSRF, CSP, throttling, safe redirects) and offline front-end assets.

2. Architecture

2.1 Layered architecture

The application follows a classical layered architecture. The browser talks HTTP to **Gunicorn**, which runs the **Flask** application. Requests are dispatched by **blueprints** to view functions; views validate input with **WTForms**, read and write data through **SQLAlchemy models**, and render **Jinja2** templates. Static assets (CSS, JavaScript, vendored libraries) are served from the same application.

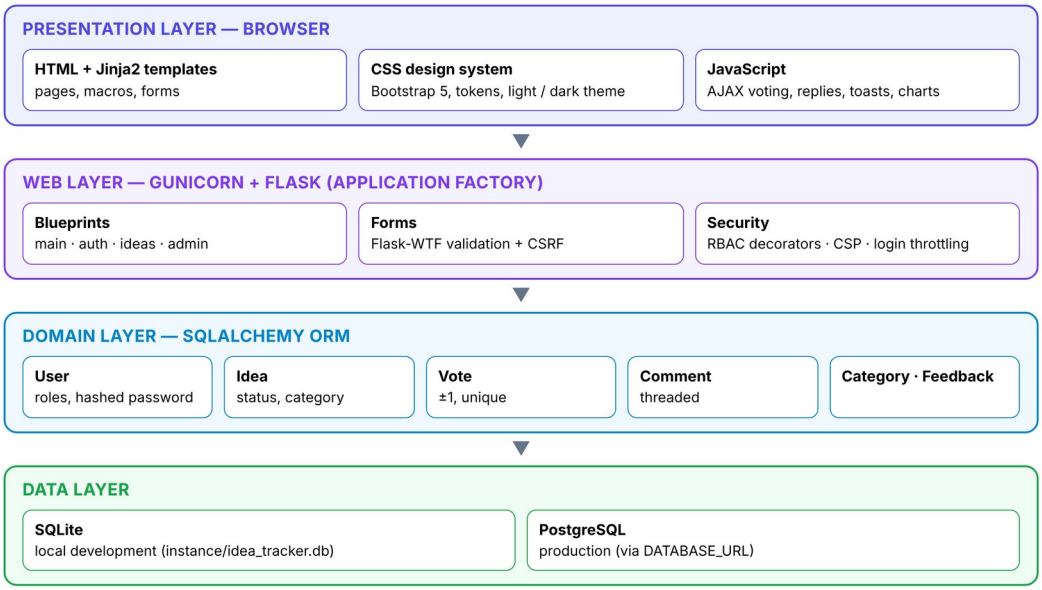


Figure 1 – Layered architecture of the application

2.2 Application factory

`create_app()` in `app/__init__.py` builds a new Flask application from a configuration class. This pattern keeps the global state minimal and lets the tests create an isolated application with an in-memory database (`TestConfig`). The factory initialises the extensions, registers the four blueprints, the template helpers, error handlers, security headers and CLI commands, then bootstraps the database.

```
def create_app(config_object=Config) -> Flask:
    app = Flask(__name__)
    app.config.from_object(config_object)

    os.makedirs(os.path.join(app.root_path, "..", "instance"), exist_ok=True)

    db.init_app(app)
    login_manager.init_app(app)
    csrf.init_app(app)

    from .blueprints.admin import bp as admin_bp
```

```
    from .blueprints.auth import bp as auth_bp
    from .blueprints.ideas import bp as ideas_bp
    from .blueprints.main import bp as main_bp

    app.register_blueprint(main_bp)
    app.register_blueprint(auth_bp)
    app.register_blueprint(ideas_bp)
    app.register_blueprint(admin_bp)

    register_template_helpers(app)
    register_error_handlers(app)
    register_security(app)
    register_cli(app)

    with app.app_context():
        bootstrap_database()

    return app
```

2.3 Request life cycle

1. The **security layer** (`register_security`) adds the response headers in an `after_request` hook, and an optional `ProxyFix` makes client IP and protocol correct behind a reverse proxy.
2. **CSRFProtect** checks the token of every state-changing request (form field or `X-CSRFToken` header for AJAX).
3. **Flask-Login** loads the user from the session; a user whose account was disabled is rejected at this point.
4. The **blueprint view** runs: role decorator → form validation → business rules → database access.
5. The **template** is rendered with auto-escaping, or a JSON response is returned for the AJAX vote.

2.4 Blueprints

Table 2 – Blueprints

Blueprint	File	Responsibility
`main`	blueprints/main.py	Home page, about, public profiles, feedback form, `/healthz`
`auth`	blueprints/auth.py	Register, login (with throttling), logout, profile and password
`ideas`	blueprints/ideas.py	List/search/sort, CRUD of ideas, votes, comments, evaluation, review queue
`admin`	blueprints/admin.py	Dashboard, users, categories, feedback inbox, CSV export

3. Project Structure

```
idea_tracker/
├── run.py                 entry point (python run.py / gunicorn run:app)
├── config.py              environment-driven configuration (Config, TestConfig)
├── seed.py                demo users and sample ideas
├── requirements.txt       runtime dependencies
├── requirements-dev.txt   + pytest
├── Procfile               gunicorn command for Heroku/Render/Beanstalk
├── Dockerfile · docker-compose.yml
├── .github/workflows/ci.yml
├── app/
│   ├── __init__.py        application factory, error handlers, CLI commands
│   ├── extensions.py      db, login_manager, csrf (created once, bound later)
│   ├── models.py          User, Category, Idea, Vote, Comment, Feedback
│   ├── forms.py           WTForms forms and validators
│   ├── security.py        CSP and headers, LoginThrottle
│   ├── utils.py           roles_required, is_safe_redirect, build_comment_tree
│   ├── blueprints/        main.py  auth.py  ideas.py  admin.py
│   ├── templates/         base.html, _macros.html, index, ideas/, auth/, admin/
│   └── static/            css/style.css · js/app.js · vendor/ (offline libs)
├── tests/                 conftest.py · test_app.py · test_v2.py (84 tests)
└── docs/                  requirements, data model, tests, deployment, guides
```

Table 3 – Size of the main modules

Module	Lines	Content
`app/models.py`	272	Six models, constants, computed columns
`app/blueprints/ideas.py`	427	Largest module: ideas, votes, comments, review
`app/blueprints/admin.py`	207	Back office
`app/__init__.py`	198	Factory, template helpers, CLI
`app/forms.py`	152	Eleven forms
`app/static/js/app.js`	228	Theme, toasts, AJAX vote, replies, charts
`app/static/css/style.css`	378	Design system

4. Configuration

`config.py` reads every setting from environment variables so that the same code runs everywhere. Two helpers deserve attention: `_database_url()` falls back to a SQLite file in `instance/` and fixes the legacy `postgres://` scheme, and `_secret_key()` generates a random key **once**, stores it in `instance/.secret_key` (mode 600, created atomically so that several workers do not race) and falls back to an ephemeral key on a read-only file system. No secret is hard-coded.

Table 4 – Configuration reference

Setting	Default	Description
`SECRET_KEY`	random, persisted	Signs sessions and CSRF tokens (set it in production)
`DATABASE_URL`	SQLite in `instance/`	Database connection URL
`SESSION_COOKIE_SECURE`	0	`1` on HTTPS (also secures the remember-me cookie)
`TRUST_PROXY`	0	`1` behind a reverse proxy
`IDEAS_PER_PAGE`	10	Pagination size
`MAX_COMMENT_LENGTH`	2000	Maximum comment length
`LOGIN_MAX_ATTEMPTS` / `LOGIN_WINDOW_SECONDS`	5 / 900	Login throttling
`WTF_CSRF_TIME_LIMIT`	8 h	Lifetime of CSRF tokens
`HOST`, `PORT`, `FLASK_DEBUG`	127.0.0.1, 5000, 0	Development server only

5. Data Model

5.1 Entity-relationship diagram

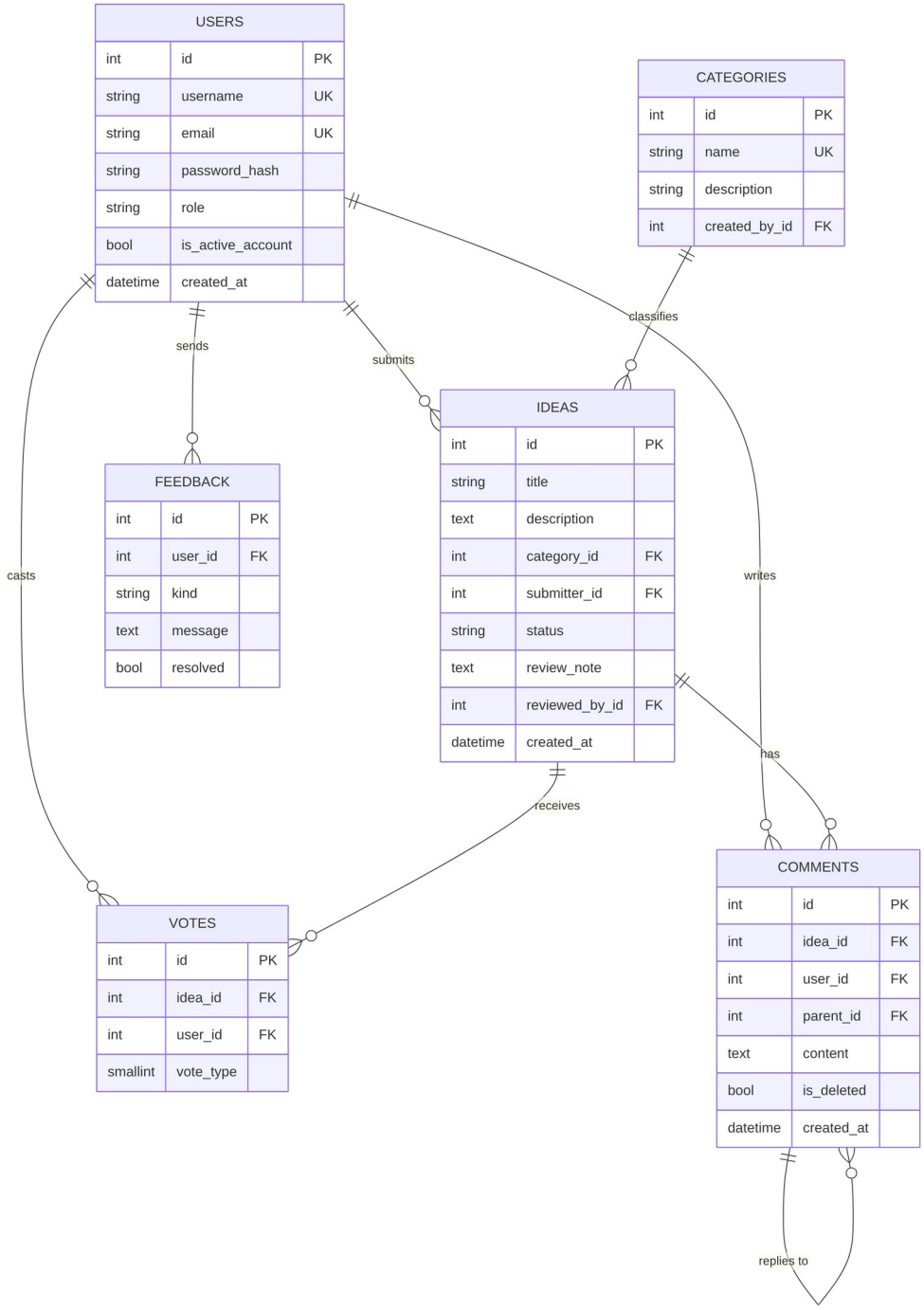


Figure 2 – Entity-relationship diagram

5.2 Tables

Table 5 – Tables and constraints

Table	Columns	Constraints

5.3 Integrity rules in the database

```
class Vote(db.Model):
    __tablename__ = "votes"

    id = db.Column(db.Integer, primary_key=True)
    idea_id = db.Column(db.Integer, db.ForeignKey("ideas.id", ondelete="CASCADE"),
nullable=False, index=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"),
nullable=False, index=True)
    vote_type = db.Column(db.SmallInteger, nullable=False)  # +1 upvote, -1 downvote
    created_at = db.Column(db.DateTime, nullable=False, default=utcnow)

    idea = db.relationship("Idea", back_populates="votes")
    user = db.relationship("User", back_populates="votes")

    __table_args__ = (
        db.UniqueConstraint("idea_id", "user_id", name="uq_vote_idea_user"),
        db.CheckConstraint("vote_type IN (-1, 1)", name="ck_vote_type"),
        ...
```

The unique constraint guarantees “one vote per user and idea” even when two requests arrive simultaneously; the CHECK constraint refuses any value other than +1 or −1. Foreign keys are enforced on SQLite with `PRAGMA foreign_keys=ON`, so development and production behave identically.

5.4 Computed columns

The score of an idea and its number of comments are **never stored**. They are correlated scalar sub-queries mapped with `column_property`, so they cannot become inconsistent and can be used in `ORDER BY` and in filters in a single SQL statement (no N+1 queries):

```
Idea.score = column_property(
    select(func.coalesce(func.sum(Vote.vote_type), 0))
    .where(Vote.idea_id == Idea.id)
    .correlate_except(Vote)
    .scalar_subquery()
)
Idea.comment_count = column_property(
    ...
```

6. Code Walkthrough

6.1 Authentication (blueprints/auth.py)

Registration validates the form (`RegisterForm`), compares usernames and e-mails case-insensitively with `func.lower(...) ==` (not `ILIKE`, whose _ and % are wildcards), hashes the password with `set_password()` (Werkzeug **scrypt**) and signs the user in. Login accepts a username or an e-mail, consults the throttle **before** checking the password, records failures, rejects disabled accounts and redirects to `next` only if `is_safe_redirect()` confirms it stays on the same host:

```
def login():
    if current_user.is_authenticated:
        return redirect(url_for("main.index"))
    form = LoginForm()
    if form.validate_on_submit():
        ident = form.identifier.data.strip()
        limit = current_app.config["LOGIN_MAX_ATTEMPTS"]
        window = current_app.config["LOGIN_WINDOW_SECONDS"]
        throttle_key = login_throttle.key(ident)

        wait = login_throttle.retry_after(throttle_key, limit, window)
        if wait:
            flash(f"Too many failed attempts. Please try again in {math.ceil(wait /
60)} minute(s).", "danger")
            return render_template("auth/login.html", form=form), 429

        user = User.query.filter(
            (db.func.lower(User.username) == ident.lower()) |
(db.func.lower(User.email) == ident.lower())
        ).first()
        if user is None or not user.check_password(form.password.data):
            login_throttle.register_failure(throttle_key)
            flash("Invalid username/email or password.", "danger")
        elif not user.is_active_account:
            flash("This account has been disabled. Please contact an administrator.",
"danger")
        else:
            login_throttle.reset(throttle_key)
            login_user(user, remember=form.remember.data)
            flash(f"Welcome back, {user.username}!", "success")
            nxt = request.args.get("next")
            return redirect(nxt if is_safe_redirect(nxt) else url_for("main.index"))
    return render_template("auth/login.html", form=form)
```

6.2 Role-based access control (utils.py)

`roles_required(*roles)` is a decorator factory. Unauthenticated visitors are sent to the login page; authenticated users with the wrong role receive HTTP 403. Shortcuts such as

`reviewer_required` and `admin_required` are built on it. Object-level rules (owner, status, self-voting, self-review) are checked inside the views because they depend on the data.

```
def roles_required(*roles):
    """Role-based access control decorator (not signed in -> login page, wrong role ->
403)."""

    def decorator(view):
        @wraps(view)
        def wrapped(*args, **kwargs):
            if not current_user.is_authenticated:
                return current_app.login_manager.unauthorized()
            if current_user.role not in roles:
                abort(403)
            return view(*args, **kwargs)

        return wrapped

    return decorator
```

6.3 Listing, searching and sorting (ideas.list_ideas)

The list view builds one SQLAlchemy `select`, adds optional filters (text, category, status) and an `ORDER BY` taken from the `SORTS` dictionary, then paginates. The search term goes through `escape_like()` so that `%` and _ are literal. The **trending** order ranks by the net votes plus the comments of the last seven days, then by overall score:

```
def _trending_order():
    """Activity in the last 7 days: votes (net) + comments, then all-time score."""
    cutoff = utcnow() - timedelta(days=7)
    recent_votes = (
        db.select(db.func.coalesce(db.func.sum(Vote.vote_type), 0))
        .where(Vote.idea_id == Idea.id, Vote.created_at >= cutoff)
        .correlate_except(Vote)
        .scalar_subquery()
    )
    recent_comments = (
        db.select(db.func.count(Comment.id))
        .where(Comment.idea_id == Idea.id, Comment.created_at >= cutoff,
Comment.is_deleted.is_(False))
        .correlate_except(Comment)
        .scalar_subquery()
    )
    return ((recent_votes + recent_comments).desc(), Idea.score.desc(),
Idea.created_at.desc())
```

6.4 Voting (ideas.vote)

The view refuses anonymous users (401 JSON for AJAX), the author of the idea and invalid values. It then creates, withdraws or switches the vote. If two simultaneous requests violate the

unique constraint, the `IntegrityError` is caught, the transaction is rolled back and the existing vote is updated instead. With the header `X-Requested-With: fetch` the answer is JSON (score, up/down counts, the user's vote); otherwise it is a redirect, so voting works without JavaScript.

```
    existing = Vote.query.filter_by(idea_id=idea.id, user_id=current_user.id).first()
    if existing is None:
        db.session.add(Vote(idea_id=idea.id, user_id=current_user.id, vote_type=value))
        my_vote = value
    elif existing.vote_type == value:  # clicking the same arrow again withdraws the
vote
        db.session.delete(existing)
        my_vote = 0
    else:
        existing.vote_type = value
        my_vote = value
    try:
        db.session.commit()
    except IntegrityError:
        # Two simultaneous clicks raced on the unique (idea, user) constraint: keep the
first.
        db.session.rollback()
        current = Vote.query.filter_by(idea_id=idea.id,
user_id=current_user.id).first()
        my_vote = current.vote_type if current else 0
    db.session.refresh(idea)
```

6.5 Threaded comments

A reply references its parent through `parent_id`; the view checks that the parent belongs to the same idea. To display a thread, all comments of the idea are loaded with one query and assembled into a forest in linear time, then rendered by a recursive Jinja macro (indentation stops growing after five levels). Deleting a comment that has replies sets `is_deleted` and blanks the text instead of removing the row.

```
def build_comment_tree(comments):
    """Turn a flat, time-ordered list of comments into a forest.

    Each comment receives a transient ``children`` list; the list of root
    comments is returned.
    """
    by_id = {c.id: c for c in comments}
    for c in comments:
        c.children = []
    roots = []
    for c in comments:
        parent = by_id.get(c.parent_id)
        if parent is not None:
            parent.children.append(c)
        else:
            roots.append(c)
    return roots
```

6.6 Evaluation workflow (ideas.review)

Only reviewers and administrators may call this endpoint, and nobody may evaluate their own idea. The view stores the status, the public note, the reviewer and the time. Quick actions of the review queue post only the status, so the existing note is preserved (`"review_note" in request.form`).

```
def review(idea_id):
    idea = db.session.get(Idea, idea_id) or abort(404)
    if idea.submitter_id == current_user.id and not current_user.is_admin:
        abort(403)  # conflict of interest: reviewers cannot judge their own idea
    form = ReviewForm()
    if form.validate_on_submit() and form.status.data in STATUSES:
        idea.status = form.status.data
        if "review_note" in request.form:  # quick actions from the queue leave the
note untouched
            idea.review_note = (form.review_note.data or "").strip()
        idea.reviewed_by_id = current_user.id
        idea.reviewed_at = utcnow()
        db.session.commit()
```

6.7 Administration (blueprints/admin.py)

- **Dashboard:** counts and group-by queries; the chart data is passed to the page as a JSON block `<script type="application/json" id="chart-data">` read by `app.js` (no inline script, see CSP).
- **Users:** search with escaped wildcards, role change, enable/disable; guards prevent changing oneself and removing the last active administrator.
- **Categories:** create, rename, delete only when no idea uses the category.
- **CSV export:** streams all ideas; `_csv_safe()` prefixes cells that start with `=`, `+`, `-` or `@` to prevent spreadsheet formula injection.

6.8 Security module (security.py)

`register_security` installs the headers and the Content-Security-Policy. Because the application serves all its assets itself and uses no inline script, the policy can be strict:

```
CSP = "; ".join(
    [
        "default-src 'self'",
        "img-src 'self' data:",
        "style-src 'self' 'unsafe-inline'",  # inline style="" attributes (progress
bars) only
        "script-src 'self'",
        "font-src 'self'",
        "connect-src 'self'",
        "object-src 'none'",
```

```
        "base-uri 'self'",
        "form-action 'self'",
        "frame-ancestors 'none'",
    ]
)
```

`LoginThrottle` keeps, per `IP|identifier` key, a deque of failure timestamps protected by a lock. `retry_after()` prunes the old entries and returns the number of seconds to wait once the limit is reached; the login view then answers HTTP 429.

```
    def retry_after(self, key: str, limit: int, window: int) -> int:
        """Seconds to wait before another attempt is allowed (0 = allowed)."""
        now = time.monotonic()
        with self._lock:
            bucket = self._failures.get(key)
            if not bucket:
                return 0
            self._prune(bucket, window, now)
            if len(bucket) < limit:
                return 0
            return max(1, int(window - (now - bucket[0])))
```

6.9 Front end

- **Templates:** `base.html` (layout, navigation, flash messages, theme), `_macros.html` (avatar, status pill, vote box, idea card, recursive comment), pages under `ideas/`, `auth/`, `admin/`.
- **`static/css/style.css`:** design tokens (CSS variables) for the light and dark themes, components (cards, vote box, stepper, sentiment bar, toasts), responsive rules, print styles.
- **`static/js/app.js`:** theme toggle and persistence, toast notifications, loading state of buttons, **AJAX voting** with the CSRF header, reply forms, character counters, back-to-top button, Chart.js charts built from the JSON block.
- **`static/js/theme-init.js`:** tiny external script loaded in `<head>` that applies the saved theme before first paint to avoid a flash.
- **`static/vendor/`:** Bootstrap, Bootstrap Icons, Chart.js and the Inter font, so that no CDN is used.

7. Routes

Table 6 – Routes and access rules

Method and path	Access	Purpose
GET /	public	Landing page, statistics
GET /about	public	About page
GET, POST /register · /login · POST /logout	public / user	Accounts
GET, POST /profile	user	E-mail and password
GET /u/<username>	public	Public profile
GET /ideas	public	List: q, category, status, sort, page
GET /my-ideas	user	Own ideas
GET, POST /ideas/new	user	Submit
GET /ideas/<id>	public	Detail and comments
GET, POST /ideas/<id>/edit · POST /ideas/<id>/delete	owner (pending) / admin	Modify
POST /ideas/<id>/vote	user	Vote, switch, withdraw (JSON or redirect)
POST /ideas/<id>/comments · POST /comments/<id>/delete	user / author / admin	Comment, reply, delete
GET /review · POST /ideas/<id>/review	reviewer, admin	Queue and evaluation
GET, POST /feedback	user	Send feedback
GET /admin/ · /admin/users · /admin/categories · /admin/feedback (+ POST actions)	admin	Back office
GET /admin/export/ideas.csv	admin	CSV export
GET /healthz	public	Liveness probe

8. Security Measures

Table 7 – Threats and countermeasures

Threat	Countermeasure
Password theft	Salted scrypt hashes (`werkzeug.security`); password rules; never logged or displayed
Brute force	`LoginThrottle`: 5 failures / 15 min per IP + account, HTTP 429
CSRF	`CSRFProtect` globally; token in forms; `X-CSRFToken` for AJAX
XSS	Jinja auto-escaping (no `|safe` on user text); CSP `script-src 'self'`, no inline scripts
SQL injection	ORM and bound parameters only; LIKE wildcards escaped
Broken access control	`roles_required` + object-level checks; disabled accounts rejected on the next request
Open redirect	`is_safe_redirect` accepts only same-host `next`
Clickjacking / sniffing	`X-Frame-Options: DENY`, `frame-ancestors 'none'`, `nosniff`
Transport / cookies	HSTS on HTTPS; HttpOnly, SameSite=Lax, optional Secure cookies
CSV injection	`_csv_safe()` neutralises `= + - @`
Weak default secret	Random key generated once, no hard-coded value

9. Installation and Setup

9.1 Prerequisites

- **Python 3.10 or later** (check with `python --version`) and `pip`.
- **Git** (optional, to clone the repository).
- For production: a PostgreSQL database and a host (see chapter 11). For containers: Docker with Compose.

9.2 Local installation

1. Get the sources and open a terminal in the project folder (`idea_tracker/`).
2. Create a virtual environment: `python -m venv .venv`.
3. Activate it: `source .venv/bin/activate` (Linux/macOS) or `.venv\Scripts\activate` (Windows).
4. Install the dependencies: `pip install -r requirements.txt`.
5. Optional – load demo data (6 users, 16 ideas): `python seed.py` (use `python seed.py --reset` to start over).
6. Start the server: `python run.py` and open `http://127.0.0.1:5000`.

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python seed.py                     # optional demo data
python run.py                      # http://127.0.0.1:5000

# development mode with auto-reload
FLASK_DEBUG=1 python run.py
```

On first start the tables and the six default categories are created automatically in `instance/idea_tracker.db`. The same can be done manually with `flask --app run.py init-db`.

9.3 Creating the first administrator

```
flask --app run.py create-admin
```

The command asks for a username, an e-mail and a password. Without demo data, this is the way to obtain an administrator who can then promote other users to reviewer.

9.4 Demonstration accounts

Table 8 – Accounts created by seed.py

Username	Role	Password
admin	administrator	Demo1234
reviewer	reviewer	Demo1234
alice, bob, carol, dave	submitter	Demo1234

These passwords are for local testing only; never seed demo data in production.

9.5 Using PostgreSQL

```
pip install -r requirements.txt            # includes psycopg2-binary
export DATABASE_URL=postgresql://user:password@localhost:5432/idea_tracker
export SECRET_KEY=$(python -c "import secrets; print(secrets.token_hex(32))")
python run.py
```

9.6 Docker

```
docker compose up --build              # app on http://localhost:8000 + PostgreSQL
docker compose exec web flask --app run.py create-admin
```

9.7 Production server

```
gunicorn run:app --workers 2 --bind 0.0.0.0:8000
```

10. Testing

10.1 Running the tests

```
pip install -r requirements-dev.txt
pytest -v                    # 84 tests
pytest tests/test_v2.py -k vote   # a subset
```

10.2 How the tests are organised

`tests/conftest.py` provides the fixtures: `app` (a fresh application with `TestConfig` – in-memory SQLite, CSRF disabled – dropped after each test), `client` (Flask test client), `users` (alice, bob, a reviewer and an administrator), `idea`, and helpers such as `make_user` and `login`. The login throttle is reset around every test. `tests/test_app.py` holds the functional tests of the first version and `tests/test_v2.py` those of the second version (security headers, throttling, review queue, profiles, export, regressions).

Table 9 – Test coverage by area

Area	Tests
Authentication and accounts	9
Ideas, categories, search and sort	11
Voting	7
Comments	7
Roles and evaluation	6
Administration	8
Feedback, CSRF, errors, health, profiles	6
Regressions of fixed bugs	5
Login throttling	3
Security headers and offline assets	5
Review queue	5
Profiles, trending, export, related ideas, bootstrap	12
Total	**84**

A GitHub Actions workflow (`.github/workflows/ci.yml`) installs the dependencies and runs the suite on Python 3.10 and 3.12 at every push. The interface was additionally checked in a real headless browser (no console errors, no CSP violations, no horizontal overflow at 390 px).

11. Deployment

The application is a WSGI app configured only through environment variables, so any host that runs Python can serve it. Step-by-step guides are in `docs/04_DEPLOYMENT.md`.

Table 10 – Hosting options

Platform	Database	How
Render (recommended)	Managed PostgreSQL	Web Service from GitHub; build `pip install -r requirements.txt`; start `gunicorn run:app`; health check `/healthz`
Heroku	Heroku Postgres	Uses the `Procfile`
PythonAnywhere	SQLite	WSGI file importing `run.app`
Docker / Compose	PostgreSQL container	`docker compose up --build`
AWS	RDS PostgreSQL	Elastic Beanstalk (Procfile) or ECS / App Runner (Dockerfile)

Production checklist

- Strong random `SECRET_KEY`; PostgreSQL instead of SQLite.
- HTTPS with `SESSION_COOKIE_SECURE=1`; `TRUST_PROXY=1` behind a proxy.
- No demo accounts; `FLASK_DEBUG` unset; regular backups.
- Create the first administrator with `flask --app run.py create-admin`.

12. Extending the Application

Table 11 – Extension recipes

Goal	What to change
Star ratings instead of up/down	Change the range of `Vote.vote_type` and the CHECK constraint; use `avg()` instead of `sum()` in `Idea.score`; adapt the vote box macro and `app.js`
New status	Add it to `STATUSES`, `STATUS_LABELS`, `STATUS_COLORS` and `STATUS_ICONS` in `models.py`
E-mail notifications	Hook into the `add_comment` and `review` views (for example with Flask-Mail)
Schema migrations	Add Flask-Migrate (Alembic) before changing tables in production
Shared rate limiting	Replace `LoginThrottle` by Flask-Limiter with a Redis backend
Another language	Add Flask-Babel; the templates already separate layout from text

Known limitations

- The login throttle is per process (multiple workers multiply the limit).
- No e-mail verification or password reset; no schema migrations yet.
- No automated test of the JavaScript; the browser checks were done with Playwright scripts.

Annex – Inventory of automated tests

#	Area	Test
1	Auth	register creates submitter and hashes password
2	Auth	register rejects weak or mismatched password
3	Auth	register rejects duplicates case insensitively
4	Auth	login logout
5	Auth	login with email and wrong password
6	Auth	disabled account cannot login
7	Auth	open redirect is blocked
8	Auth	protected pages redirect to login
9	Auth	change password
10	Ideas	default categories exist
11	Ideas	submit idea with existing category
12	Ideas	submit idea creates dynamic category once
13	Ideas	idea validation
14	Ideas	anonymous can read but not write
15	Ideas	xss is escaped
16	Ideas	owner edit delete rules
17	Ideas	admin can delete any idea and cascade cleans up
18	Ideas	my ideas only lists own
19	Ideas	search filter sort pagination
20	Ideas	sort by top
21	Voting	upvote change and withdraw
22	Voting	one vote per user and scores add up
23	Voting	cannot vote on own idea
24	Voting	anonymous vote returns 401 json
25	Voting	invalid vote value
26	Voting	vote unknown idea 404
27	Voting	non ajax vote redirects
28	Comments	post comment and threaded reply
29	Comments	reply to comment of other idea is rejected
30	Comments	empty and overlong comments rejected
31	Comments	delete rules
32	Comments	deleting parent keeps thread
33	Comments	admin can delete any comment

#	Area	Test
34	Comments	comment html is escaped
35	Roles	reviewer can evaluate
36	Roles	submitter cannot evaluate
37	Roles	anonymous review redirects to login
38	Roles	reviewer cannot review own idea
39	Roles	invalid status rejected
40	Roles	review form only shown to reviewers
41	Admin	admin area requires admin
42	Admin	change role
43	Admin	admin cannot demote or disable self
44	Admin	cannot disable last active admin
45	Admin	disable toggle
46	Admin	disabled user session is invalidated
47	Admin	category crud and protection
48	Admin	dashboard counts
49	FeedbackAndMisc	feedback roundtrip
50	FeedbackAndMisc	feedback too short
51	FeedbackAndMisc	home page and 404 and health
52	FeedbackAndMisc	security headers
53	FeedbackAndMisc	profile email update and conflict
54	CSRF	csrf enforced when enabled
55	BugFixes	underscore username is not a wildcard clash
56	BugFixes	percent in email is not a wildcard
57	BugFixes	admin user search escapes wildcards
58	BugFixes	no wtforms deprecation warnings
59	BugFixes	secret key is not the old hardcoded default
60	LoginThrottle	blocks after repeated failures then recovers
61	LoginThrottle	success resets counter
62	LoginThrottle	throttle is per account
63	SecurityHeaders	csp and friends
64	SecurityHeaders	pages have no inline or external scripts
65	SecurityHeaders	admin dashboard has only json data block
66	SecurityHeaders	vendored assets are served
67	SecurityHeaders	hsts only on https
68	ReviewQueue	requires reviewer

#	Area	Test
69	ReviewQueue	lists pending oldest first and quick approve
70	ReviewQueue	quick action keeps existing note
71	ReviewQueue	review redirect rejects external next
72	ReviewQueue	nav badge shows pending count for reviewers only
73	PublicProfile	profile stats
74	PublicProfile	unknown user 404
75	PublicProfile	profile is xss safe
76	LeaderboardAndTrending	home lists contributors
77	LeaderboardAndTrending	trending prefers recent activity
78	ExportAndMisc	csv export admin only and safe
79	ExportAndMisc	about page
80	ExportAndMisc	detail shows related ideas and stepper
81	ExportAndMisc	vote buttons disabled on own idea
82	ExportAndMisc	status filter tabs
83	ExportAndMisc	bootstrap is idempotent
84	ExportAndMisc	error pages render with layout